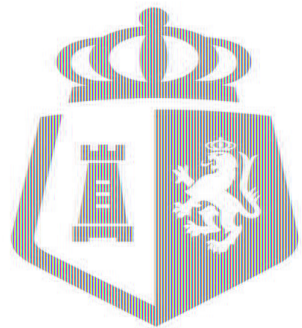




BPI

Java Exception Handling & Logging with SLF4J + Logback

A BPI Training Program

Topics

Part 1: Exception Handling

- What is Exception
- Checked vs Unchecked Exception
- try-catch-finally
- throw vs throws
- Custom exceptions
- Practice exercise

Part 2: Logging

- Logging with SLF4J + Logback
- Practice exercise

Learning Objectives

By the end of this session, you will:

- . Understand what exceptions are and how they work
- . Know the difference between checked and unchecked exceptions
- . Master try-catch-finally blocks
- . Create custom exceptions for your needs
- . Implement professional logging
- . Apply industry best practices

PART 1: Exception Handling

Important Rule:

"Your program should NEVER crash unexpectedly"

In banking: One error should not stop the whole system!

What Happens When Things Go Wrong?

Scenario:

- Customer enters invalid account number
- Account doesn't exist
- Network connection fails
- Daily withdrawal limit exceeded
- Insufficient funds

Question: How should your program respond?

Without Exception Handling

Code Example:

```
public class WithoutExceptionHandling {  
    public static void main(String[] args) {  
        int result = 10 / 0;  
  
        System.out.println("Result: " + result);  
        System.out.println("Program continues...");  
    } // end of main method  
} // end of WithoutExceptionHandling
```

Result:

- Program crashes immediately
- User sees confusing error message
- Remaining code never executes
- Poor user experience

With Exception Handling

Code Example:

```
public class WithExceptionHandling {  
    public static void main(String[] args) {  
        try {  
            int result = 10 / 0;  
            System.out.println("Result: " + result);  
        } catch (ArithmeticException e) {  
            System.out.println("Cannot divide by zero!");  
        }  
  
        System.out.println("Program continues gracefully...");  
    } // end of main method  
  
} // end of WithExceptionHandling
```

Result:

- Program catches the error
- Shows friendly message
- Continues executing
- Professional behavior



Code Demo

What is an Exception?

Definition:

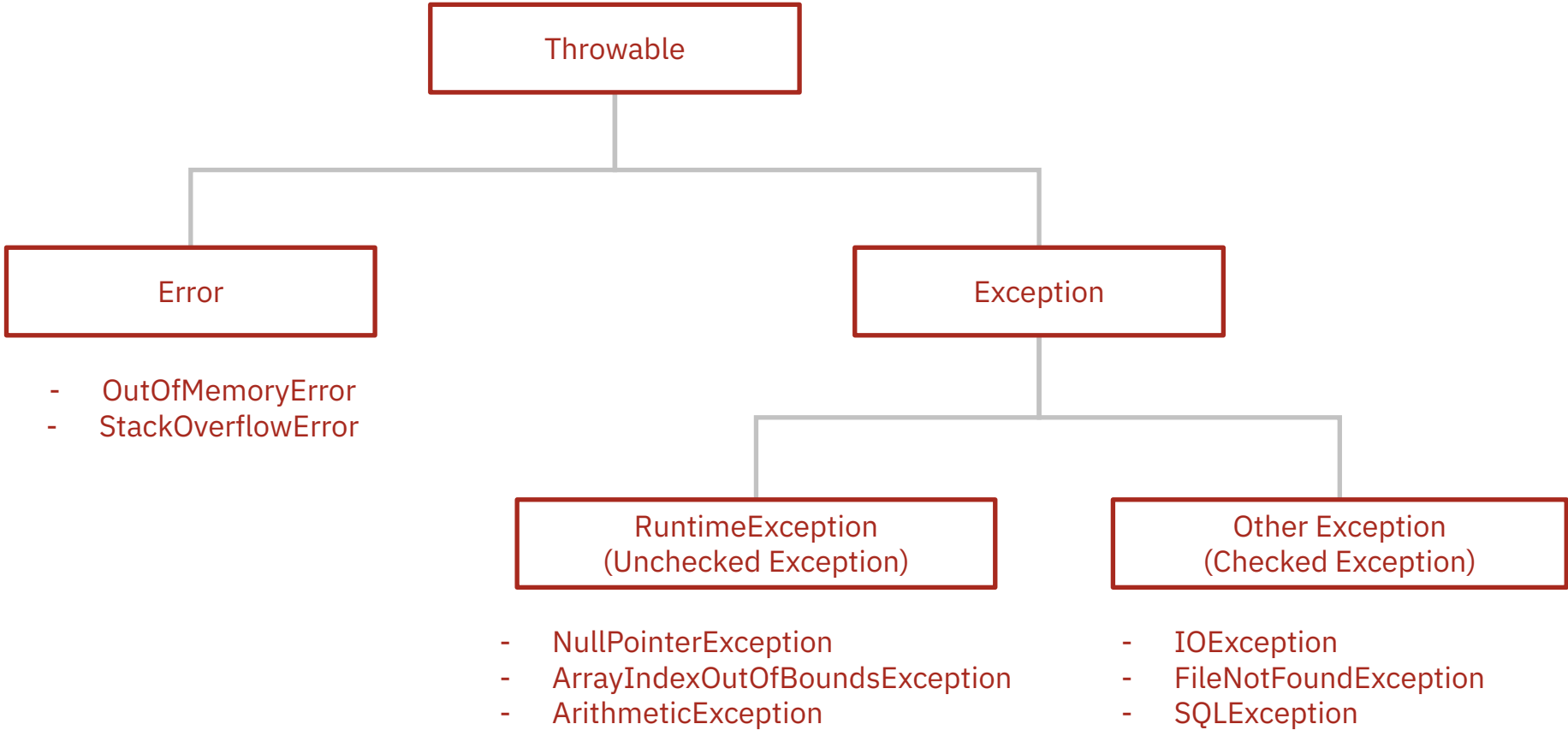
An exception is something unexpected that happens while your program is running.

Important Points:

- It's an EVENT (something that happens)
- It disrupts the normal flow
- It's an OBJECT (yes, like classes you learned!)
- It's NOT a bug - it's an EXPECTED problem



Exception Hierarchy



Unchecked vs Checked Exceptions

Unchecked Exceptions:

- Programming errors
- Compiler doesn't force you to handle

Examples:

- NullPointerException
- ArithmeticException
- ArrayIndexOutOfBoundsException

Easy way to remember:

Unchecked = Your mistake (fix your code)

Checked = External problem (must handle it)

Checked Exceptions:

- External problems (outside your control)
- Compiler FORCES you to handle

Examples:

- IOException
- FileNotFoundException
- SQLException

The try-catch Block

Syntax (How to write it):

```
try {  
    // Contains code that MIGHT throw an exception  
    // The "risky" code that could fail  
} catch (ExceptionType e) {  
    // ExceptionType = Specific exception name (NullPointerException,  
    // IOException, etc.)  
    // e = Variable name (you choose this - 'e' or 'exception' is common)  
    // Put code to handle the exception here  
    // This runs ONLY if an exception happens  
}
```

How it works:

- Step 1: Java tries to run the code inside "try"
- Step 2: If exception happens → jumps to "catch"
- Step 3: If no exception → skips "catch" completely
- Step 4: Program continues normally after

Think of it as: TRY this, but if it fails, CATCH the problem



All Built-in Exception Types - Quick Reference

ExceptionType	Type	When It Happens
Exception	Checked	Catches everything - making it too generic for common use.
NullPointerException	Unchecked	Using null object/variable
ArithmeticException	Unchecked	Invalid math (divide by zero)
ArrayIndexOutOfBoundsException	Unchecked	Array position doesn't exist
NumberFormatException	Unchecked	Can't convert text to number
IllegalArgumentException	Unchecked	Method gets invalid argument
IllegalStateException	Unchecked	Method called at wrong time
ClassCastException	Unchecked	Wrong type conversion
RuntimeException	Unchecked	General runtime error
IndexOutOfBoundsException	Unchecked	Invalid index position
StringIndexOutOfBoundsException	Unchecked	String position doesn't exist



All Built-in Exception Types - Quick Reference

ExceptionType	Type	When It Happens
IOException	Checked	Input/Output operation fails
FileNotFoundException	Checked	File doesn't exist or can't open
SQLException	Checked	Database operation fails
ClassNotFoundException	Checked	Can't find class definition
NoSuchMethodException	Checked	Method not found in class
NoSuchFieldException	Checked	Field/variable not found in class
ParseException	Checked	Can't parse string format
InterruptedException	Checked	Thread interrupted while waiting
MalformedURLException	Checked	Invalid URL format
EOFException	Checked	End of file reached unexpectedly



Exception Object

When you catch an exception:

```
catch (ArithmeticException e) { ... }
```

Common Exception Methods You'll Use:

Method	Returns	Example	Use
getMessage()	Short error description (String)	"/ by zero"	When you want just the error message
toString()	Exception name + message (String)	"java.lang.ArithmeticException: / by zero"	When you want exception type and message together
printStackTrace()	Nothing (prints to console)	Full error trace - where error happened, which line, which method	During development/debugging
getCause()	The original exception that caused this one		When exceptions are chained (advanced)



Code Demo

SEATWORK #1: Your Turn to Try! (Branch: M4_Activity1)

Time: 10 minutes

Your Task: Display account name safely

Instructions:

1. Create a `class` called `BankAccount`
2. Create a method: `getAccountName(String accountNumber)`
 - If `accountNumber` equals "ACC-001", return "Juan Dela Cruz"
 - If `accountNumber` equals "ACC-002", return "Maria Santos"
 - Otherwise, return null
3. Create the `testCase` method:
 - Method signature: `public static void testCase(String accountNumber)`
 - Print which account is being looked up
 - Inside a `try` block:
 - Call `getAccountName(accountNumber)` and assign to variable `String name`
 - Convert `name` to uppercase using `name.toUpperCase()`
 - Print the uppercase name
 - Inside a `catch` block:
 - Catch `NullPointerException`
 - Print "Error: Account not found!"
4. In the `main` method:
 - Print a header message
 - Call `testCase("ACC-001")` to test a valid account
 - Call `testCase("ACC-999")` to test an invalid account (triggers exception)
 - Print a completion message

Expected Output:

```
=== Bank Account Name Display ===
```

```
Looking up account: ACC-001
```

```
Account holder: JUAN DELA CRUZ
```

```
Looking up account: ACC-999
```

```
Error: Account not found!
```

```
=== Program completed successfully! ===
```

Multiple catch Blocks

Purpose:

- Handle different types of exceptions in different ways
- Provide specific, helpful error messages for each problem type
- Take different recovery actions based on the specific error

Syntax:

```
try {  
    // Code that might cause DIFFERENT exceptions  
} catch (ExceptionType1 e) {  
    // Handle problem type 1  
} catch (ExceptionType2 e) {  
    // Handle problem type 2  
} catch (ExceptionType3 e) {  
    // Handle problem type 3  
}
```

Important Rule: ORDER MATTERS!

- Put specific exceptions first
- Put general exceptions last
- Java checks from top to bottom

Why? Because Java uses the FIRST match it finds



Code Demo

SEATWORK #2: Your Turn to Try! (Branch: M4_Activity2)

Time: 10 minutes

Your Task: Create a simple ATM withdrawal system

Instructions:

1. Create a **class** called **ATMSystem**
2. Create an array of account balances:
 - **double[] accounts = {10000, 15000, 20000};**
3. Create method: **processWithdrawal(String accountIndex, String amountInput)**
 - Use try with TWO catch blocks
 - Inside try:
 - Parse **accountIndex** to int (use **Integer.parseInt()**)
 - Get balance from array: **accounts[index]**
 - Parse **amountInput** to double (use **Double.parseDouble()**)
 - Check if **amount > balance**, print **"Insufficient funds"**
 - Otherwise, calculate new balance and print success
4. Use TWO catch blocks:
 - **catch (NumberFormatException e) → "Invalid input"**
 - **catch (ArrayIndexOutOfBoundsException e) → "Account not found"**
 - **catch (Exception e) → "Transaction failed"** (catch-all for unexpected errors)
5. Test FOUR scenarios in main:
 - Account **"1"**, Amount **"5000"** → Success
 - Account **"abc"**, Amount **"5000"** → **NumberFormatException**
 - Account **"10"**, Amount **"5000"** → **ArrayIndexOutOfBoundsException**
 - Account **"1"**, Amount **"20000"** → **Insufficient funds**

Expected Output:

```
=== ATM Withdrawal System ===
```

```
--- Test 1: Valid Withdrawal ---
```

```
Account=1, Amount=5000
```

```
Current balance: ₱15000.00
```

```
Withdrawal: ₱5000.00
```

```
New balance: ₱10000.00
```

```
Withdrawal successful!
```

```
--- Test 2: Invalid Account Index ---
```

```
Account=abc, Amount=5000
```

```
Error: Invalid input!
```

```
Please enter valid numbers.
```

```
--- Test 3: Account Not Found ---
```

```
Account=10, Amount=5000
```

```
Error: Account not found!
```

```
Invalid account index.
```

```
--- Test 4: Insufficient Funds ---
```

```
Account=1, Amount=20000
```

```
Current balance: ₱15000.00
```

```
Withdrawal: ₱20000.00
```

```
Insufficient funds! Cannot withdraw
```

```
₱20000.00
```

```
=== All tests completed! ===
```

The finally Block (The "No Matter What" Block)

The `finally` Block - ALWAYS Runs!

Syntax:

```
try {  
    // Risky code  
} catch (ExceptionType e) {  
    // Handle error  
} finally {  
    // This ALWAYS runs - no matter what!  
    // Use for cleanup: close files,  
    // disconnect from database, etc.  
}
```

When does finally run?

- If no exception → finally runs
- If exception happens and caught → finally runs
- If exception happens and NOT caught → finally STILL runs (then program crashes)

Think of it as: "No matter what happens, do this!"



Code Demo

SEATWORK #3: Your Turn to Try! (Branch: M4_Activity3)

Time: 10 minutes

Your Task: ATM Balance Check with Receipt Printing

Instructions:

1. Create a class called `ATMTransaction`
2. Create method: `checkBalance(String accountNumber, double balance)`
 - Inside the method:
 - Try Block:
 - Print `"Processing balance inquiry..."`
 - Validate account number format by converting to integer:
`Integer.parseInt(accountNumber)`
 - Get account type from first character: `char accountType = accountNumber.charAt(0)`
 - Determine account type name:
 - If `accountType == '1'` → Print `"Account Type: Savings"`
 - If `accountType == '2'` → Print `"Account Type: Checking"`
 - Otherwise → Print `"Account Type: Unknown"`
 - Print `"Account Number: " + accountNumber`
 - Print `"Current Balance: ₱" + balance`
 - Print `"Balance inquiry successful!"`
 - Write TWO catch blocks:
 - `catch (NumberFormatException e) → Print "Error: Invalid account number format! Account numbers must be numeric."`
 - `catch (StringIndexOutOfBoundsException e) → Print "Error: Account number is empty or invalid!"`
 - Write finally block (ALWAYS executes):
 - Print `"\n===== RECEIPT ====="`
 - Print `"Transaction Date: December 3, 2025"`
 - Print `"Transaction Type: Balance Inquiry"`
 - Print `"ATM Location: Main Branch"`
 - Print `"Thank you for banking with us!"`
 - Print `"===== \n"`
3. In main method, test FOUR scenarios:
 - `checkBalance("100123456", 15000.00) → Valid Savings account (success)`
 - `checkBalance("200987654", 25000.00) → Valid Checking account (success)`
 - `checkBalance("ABC12345", 15000.00) → NumberFormatException (invalid format)`
 - `checkBalance("", 15000.00) → StringIndexOutOfBoundsException (empty string)`



SEATWORK #3: Your Turn to Try! (Branch: M4_Activity3)

```
Expected Output:
=== ATM BALANCE INQUIRY SYSTEM ===

--- Test Case 1: Valid Savings Account ---
Processing balance inquiry...
Account Type: Savings
Account Number: 100123456
Current Balance: ₱15000.0
Balance inquiry successful!

===== RECEIPT =====
Transaction Date: December 3, 2025
Transaction Type: Balance Inquiry
ATM Location: Main Branch
Thank you for banking with us!
=====

--- Test Case 2: Valid Checking Account ---
Processing balance inquiry...
Account Type: Checking
Account Number: 200987654
Current Balance: ₱25000.0
Balance inquiry successful!

===== RECEIPT =====
Transaction Date: December 3, 2025
Transaction Type: Balance Inquiry
ATM Location: Main Branch
Thank you for banking with us!
=====
```

```
--- Test Case 3: Invalid Account Number Format ---
Processing balance inquiry...
Error: Invalid account number format! Account numbers must be
numeric.

===== RECEIPT =====
Transaction Date: December 3, 2025
Transaction Type: Balance Inquiry
ATM Location: Main Branch
Thank you for banking with us!
=====

--- Test Case 4: Empty Account Number ---
Processing balance inquiry...
Error: Invalid account number format! Account numbers must be
numeric.

===== RECEIPT =====
Transaction Date: December 3, 2025
Transaction Type: Balance Inquiry
ATM Location: Main Branch
Thank you for banking with us!
=====
```


throw vs throws

throw **vs** throws - **DON'T CONFUSE THEM!**

throws **(with 's')**:

- Used in METHOD SIGNATURE
- DECLARES that method might throw exception
- Doesn't create anything, just warns

Example:

```
public void method() throws IOException {  
    // your code here...  
}
```

throw **(without 's')**:

- Used INSIDE method body
- Actually THROWS an exception NOW
- Creates exception object

Example:

```
throw new Exception("Error!");
```



Code Demo



SEATWORK #4: Your Turn to Try! (Branch: M4_Activity4)

Time: 10 minutes

Your Task: Create account number validator

Instructions:

1. Create a class `AccountValidator`
2. Create method: `validateAccountNumber(String accountNumber)`
 - In method signature, add: `throws Exception`
 - Inside method:
 - If `accountNumber` is `null` → `throw NullPointerException`
`"Cannot be null"`
 - If length is not `10` → `throw Exception` `"Must be 10 digits"`
 - If valid → `print "Valid account: [number]"`
3. In main, test THREE cases:
 - `"1234567890"` (valid - should pass)
 - `"123"` (invalid - too short)
 - `null` (invalid - null)
4. Use `try-catch` to handle exceptions

Expected Output:

Valid account: 1234567890

Error: Must be 10 digits

Error: Cannot be null



Custom Exception

Make errors more specific and meaningful

- Built-in exceptions are too generic: "Exception" doesn't tell you WHAT went wrong
- Custom exceptions give exact problem: "InsufficientFundsException" immediately tells you the issue

Match your business/application rules

- Your app has unique rules that Java doesn't know about
- Example: Java has no "DailyLimitExceededException" - but your banking app needs it!

Provide better error messages and debugging

- Include specific details: account number, shortage amount, limit exceeded
- Help developers fix problems faster

Organize exception handling by type

- Catch specific problems differently
- Handle "OutOfStockException" differently than "InvalidDiscountCodeException"

Naming rule: Always end with "Exception"

Example names:

- InsufficientFundsException
- InvalidAccountNumberException
- DailyLimitExceededException

Template:

```
public class YourExceptionName extends Exception {  
  
    public YourExceptionName (String message) {  
        super (message) ;  
    }  
}  
  
public class YourExceptionName extends RuntimeException  
{  
  
    public YourExceptionName (String message) {  
        super (message) ;  
    }  
}
```



Code Demo

SEATWORK #5: Your Turn to Try! (Branch: M4_Activity5)

Time: 10 minutes

Your Task: Create an account number validator using TWO different custom exceptions

Instructions:

Step 1: Create TWO Custom Exception Classes

- Exception 1 `InvalidAccountNumberException` - CHECKED Exception (extends `Exception`)
- Exception 2 `InvalidAccountFormatException` - UNCHECKED Exception (extends `RuntimeException`)

Step 2: Create a class named `AccountValidator`

Step 3: Create method `validateAccountNumber(String accountNumber)`

- In method signature, add: `throws InvalidAccountNumberException`

Note: Only `InvalidAccountNumberException` is declared in throws (checked exception). The unchecked exceptions don't need to be declared!

- Inside method, implement these validations IN ORDER:
 - Check for null:
 - If `accountNumber` is null → throw `NullPointerException("Account number cannot be null")`
 - Check format:
 - If contains non-digits → throw `InvalidAccountFormatException("Account number must contain only digits")`
 - **Hint:** Use for loop `for (char c : accountNumber.toCharArray())` with `if (!Character.isDigit(c))` then throw
 - Check length:
 - If `length != 10` → throw `InvalidAccountNumberException("Account number must be exactly 10 digits")`
 - 4. If valid:
 - Print: `"Valid account number: [accountNumber]"`

Step 4: Create helper method `testValidation(String testName, String accountNumber)` to avoid repeating try-catch blocks

- Inside method:
 - Print the test name
 - Call `validateAccountNumber` inside try block
 - Catch three exceptions: `InvalidAccountNumberException`, `InvalidAccountFormatException`, `NullPointerException`
 - Print error messages

Step 5: In `main()` method, test FIVE cases

Use the `testValidation` helper method to test:

1. `"1234567890"` - valid (10 digits)
2. `"123"` - too short
3. `"12345ABC90"` - contains letters
4. `"1234 567890"` - contains space
5. `null` - null value

Expected Output:

```
=== Account Number Validation Test ===
```

```
Test 1: Valid account (1234567890)
Valid account number: 1234567890
```

```
Test 2: Too short (123)
Error: Account number must be exactly 10
digits
```

```
Test 3: Contains letters (12345ABC90)
Warning: Account number must contain only
digits
```

```
Test 4: Contains space (1234 567890)
Warning: Account number must contain only
digits
```

```
Test 5: Null value
Warning: Account number cannot be null
```

PART 1 Summary - What We Learned!

What we learned:

- ✓ What exceptions are (unexpected events)
- ✓ Exception types (checked vs unchecked)
- ✓ Common exception names (NullPointerException, IOException, etc.)
- ✓ How to use try-catch-finally
- ✓ throw vs throws (the 's' matters!)
- ✓ Creating custom exceptions

Key Points to Remember:

- try = attempt risky code
- catch = handle the error (use specific exception names!)
- finally = always runs (cleanup)
- throws = warning in method signature
- throw = actually throw the exception
- Custom exceptions = clear, specific errors